RUNGTA INTERNATIONAL SKILLS UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING IN SPECIALIZATION WITH DATA SCIENCE IN ASSOCIATION WITH GOOGLE

PROGRAMMING FUNDAMENTALS AND LOGIC BUILDING USING PYTHON

Project Report ON

PS No. 87 - ATM Management System

Submitted By:

Name: Gyanvi Yadav

ERP No: RU-26-00255

Class: 1C

FIRST SEMESTER

Submitted To:

Shreyash Khedekar Sir

PROBLEM STATEMENT

RUNGTA INTERNATIONAL SKILLS UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Bachelor of Technology Computer Science and Engineering in association with IBM
First Semester

<table>
<tr><td>PS No.</td><td>87</td></tr>
<tr><td>Project Title</td><td>ATM Management System</td></tr>
<tr><td>Problem Statement</td><td>Create a menu-driven ATM simulation where a user logs in with a PIN and can check balance, deposit, withdraw and view a mini statement. Account data and transactions are stored in a file.
Language: Python 3 Concepts: Conditions, Loops, Functions, Exceptions, File Handling
Note: Default PIN is 1234 and opening balance is 10000 (stored in atm.txt). Each test case starts with this default data.</td></tr>
<tr><td>Test Cases / Case Statements (4 cases)</td><td>
<table>
<tr><th>INPUT</th><th>EXPECTED OUTPUT</th></tr>
<tr><td colspan="2">Test Case 1: Normal Input</td></tr>
<tr><td>PIN: 1234
Menu option 2 (Deposit)
Amount: 5000</td><td>Login successful
Deposit successful. New Balance: 15000</td></tr>
<tr><td colspan="2">Test Case 2: Boundary / Minimum Input</td></tr>
<tr><td>PIN: 1234
Menu option 3 (Withdraw)
Amount: 10000 (equal to balance)</td><td>Withdrawal successful. New Balance: 0</td></tr>
<tr><td colspan="2">Test Case 3: Invalid Input</td></tr>
<tr><td>PIN: 1111, then 2222, then 3333</td><td>Invalid PIN. Attempts left: 2
Invalid PIN. Attempts left: 1
Account locked. Exiting...</td></tr>
<tr><td colspan="2">Test Case 4: Different Valid Scenario</td></tr>
<tr><td>PIN: 1234
(a) Option 3 (Withdraw): abc
(b) Option 3 (Withdraw): 2000
(c) Option 4 (Mini statement)</td><td>(a) Invalid amount. Please enter numbers only.
(b) Withdrawal successful. New Balance: 8000
(c) 1. Withdraw 2000 | Balance: 8000</td></tr>
</table>
</td></tr>
<tr><td>Bare Minimum Requirements</td><td>1. Write the solution in Python 3 (one .py file) and run it from the console.
2. Verify the PIN with a maximum of 3 attempts using a loop.
3. Implement balance check, deposit, withdraw and mini statement using functions.
4. Handle invalid input and file errors using try-except.
5. Save balance and transactions to a file after every operation.</td></tr>
<tr><td>Instruction</td><td>1. Understand the problem statement and define the problem clearly.
2. Develop the solution using proper logic and programming concepts.
3. Implement all required features and pass the given test cases.
4. Test and debug the project to ensure correct output.
5. Prepare the documentation/report and upload the code and report to GitHub.
6. Complete and submit the project within the given deadline.</td></tr>
</table>

Introduction

The ATM Management System is a menu-driven program that works like a real ATM machine. The user logs in using a PIN. After successful login, the user can check balance, deposit money, withdraw money and view mini statement. All data is stored in atm.txt file so that the balance is saved even after the program is closed.

Objective

The main objective of this project is to implement concepts of Python like Conditional Statements, Loops, Functions, Exception Handling and File Handling in a real-life application.

CODE

```
# ATM Management System

FILE_NAME = "atm.txt"
DEFAULT_PIN = "1234"
DEFAULT_BALANCE = 10000

def load_data():
    try:
        with open(FILE_NAME, "r") as f:
            lines = f.read().strip().split("\n")
            pin = lines[0].strip()
            balance = int(lines[1].strip())
            transactions = lines[2:] if len(lines) > 2 else []
            return pin, balance, transactions
    except (FileNotFoundError, IndexError,
ValueError):
        with open(FILE_NAME, "w") as f:

f.write(f"{DEFAULT_PIN}\n{DEFAULT_BALANCE}\n")
        return DEFAULT_PIN, DEFAULT_BALANCE, []

def save_data(pin, balance, transactions):
    try:
        with open(FILE_NAME, "w") as f:
            f.write(f"{pin}\n{balance}\n")
            for t in transactions:
```

```
                f.write(t + "\n")
    except Exception as e:
        print(f"File error: {e}")

def check_balance(balance):
    print(f"Current Balance: {balance}")

def deposit(balance, transactions):
    try:
        amount = int(input("Enter amount to deposit: "))
        if amount <= 0:
            print("Invalid Amount. Please enter numbers
only.")
            return balance
        balance += amount
        transactions.append(f"Deposit {amount} |
Balance: {balance}")
        print(f"Deposit Successful. New Balance:
{balance}")
    except ValueError:
        print("Invalid Amount. Please enter numbers
only.")
    return balance

def withdraw(balance, transactions):
    try:
        amount = input("Enter amount to withdraw: ")
        amount = int(amount)
        if amount <= 0:
```

```
            print("Invalid Amount. Please enter numbers
only.")
            return balance
        if amount > balance:
            print("Insufficient Balance.")
            return balance
        balance -= amount
        transactions.append(f"Withdraw {amount} |
Balance: {balance}")
        print(f"Withdrawal Successful. New Balance:
{balance}")
    except ValueError:
        print("Invalid amount. Please enter numbers
only.")
    return balance

def mini_statement(transactions):
    if not transactions:
        print("No Transactions Yet.")
    else:
        print("--- MINI STATEMENT ---")
        for i, t in enumerate(transactions, 1):
            print(f"{i}. {t}")

def main():
    pin, balance, transactions = load_data()



    attempts = 3
```

```
    logged_in = False
    while attempts > 0:
        entered_pin = input("Enter PIN: ")
        if entered_pin == pin:
            print("Login Successful")
            logged_in = True
            break
        else:
            attempts -= 1
            if attempts == 0:
                print("Account Locked. Exiting...")
                return
            else:
                print(f"Invalid PIN. Attempts left:
{attempts}")


    while True:
        print("\n--- ATM Menu ---")
        print("1. Check Balance")
        print("2. Deposit")
        print("3. Withdraw")
        print("4. Mini Statement")
        print("5. Exit")
        try:
            choice = int(input("Enter your choice: "))
        except ValueError:
            print("Invalid Amount. Please enter numbers
only.")
```

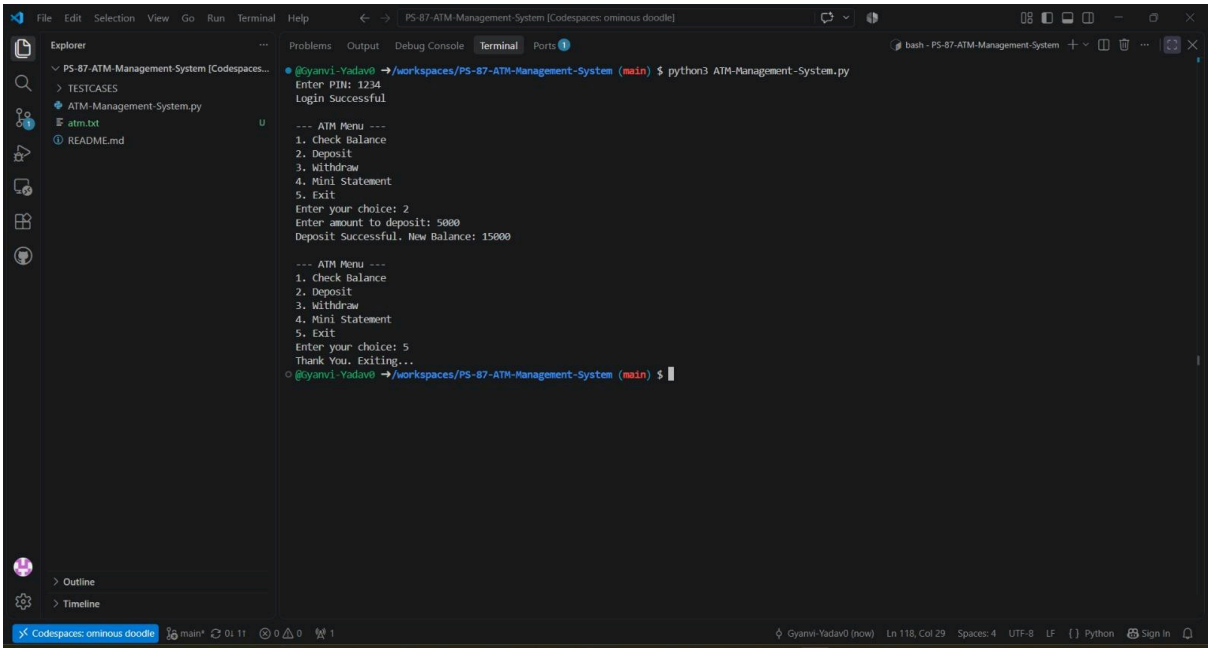
```
            continue

        if choice == 1:
            check_balance(balance)
        elif choice == 2:
            balance = deposit(balance, transactions)
            save_data(pin, balance, transactions)
        elif choice == 3:
            balance = withdraw(balance, transactions)
            save_data(pin, balance, transactions)
        elif choice == 4:
            mini_statement(transactions)
        elif choice == 5:
            print("Thank You. Exiting...")
            break
        else:
            print("Invalid Choice.")

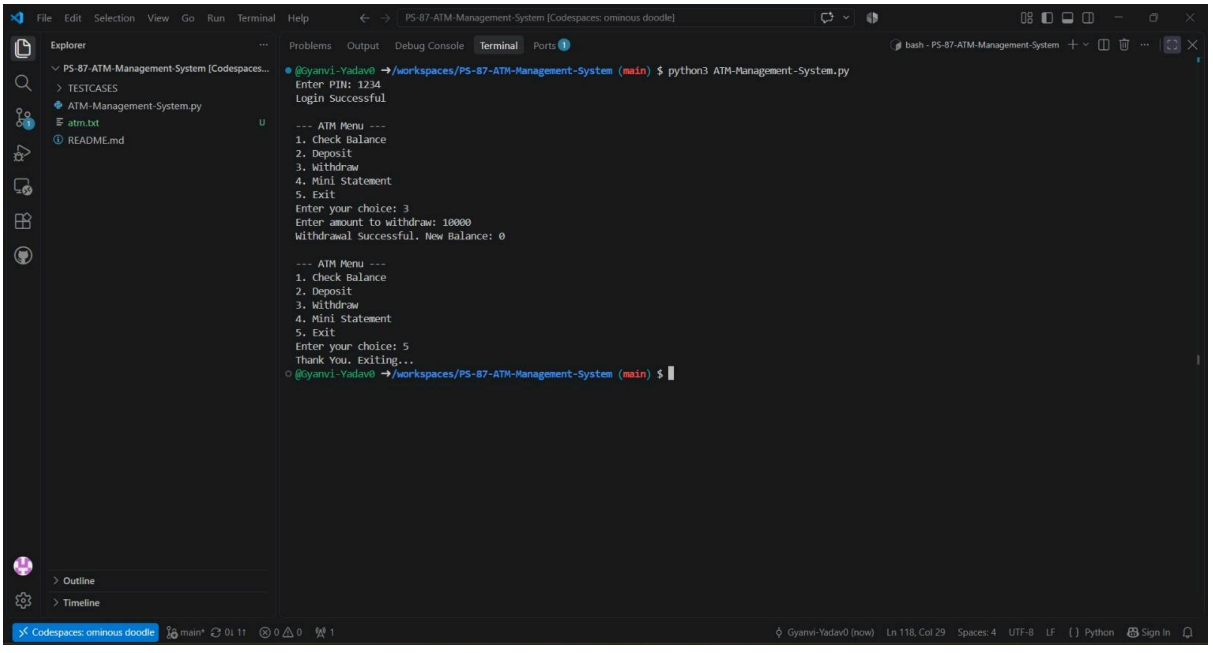
if __name__ == "__main__":
    main()
```

TESTCASES

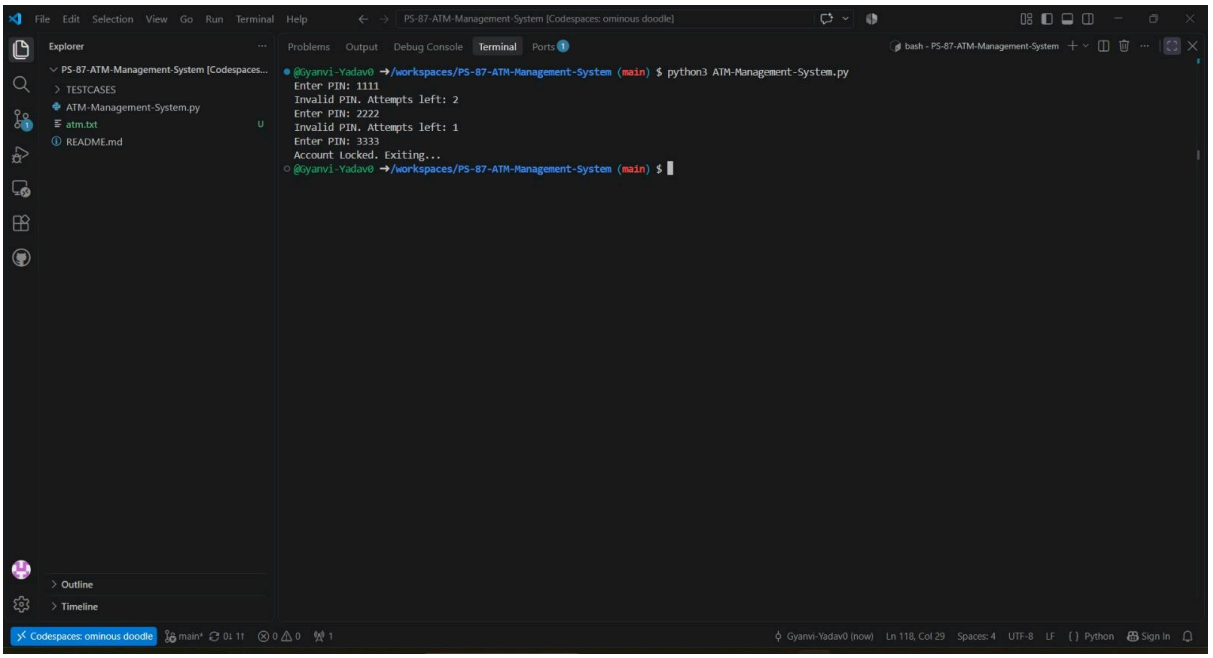
Testcase-1 (Normal Input)



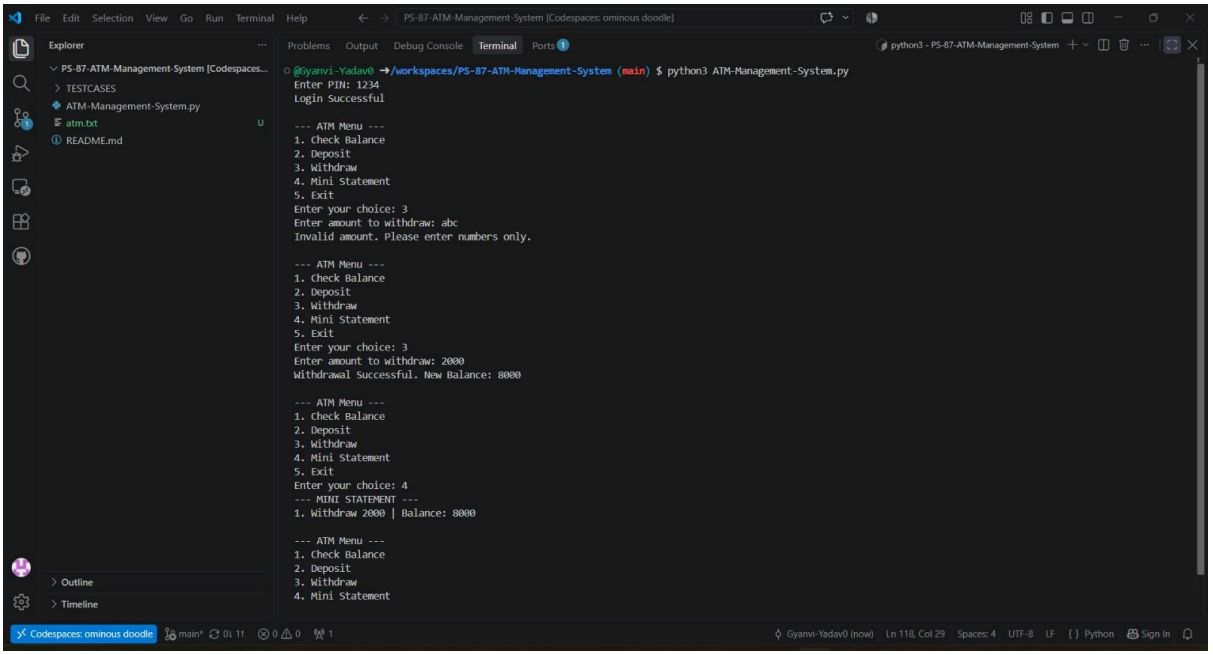
Testcase-2 (Boundary/Minimum Input)



Testcase-3 (Invalid Input)



Testcase-4 (Different Valid Scenario)



Conclusion

In this project, I have successfully developed an ATM Management System using Python. I have learned File Handling, Functions and Exception Handling. All 4 Test Cases have passed successfully.
